
PIPELINE COMPLÈTE : DÉTECTION → RÉOLUTION → VENTE
Transformer un problème observé en produit ou service
vendu avec Python comme outil

« Le développeur freelance ne vend pas du code.
Il vend la résolution d'un problème dont il a su
reconnaître l'existence avant tout le monde. »

STRUCTURE DE CETTE PIPELINE :

PHASE 1 → Détecter le problème
PHASE 2 → Qualifier et mesurer
PHASE 3 → Prototyper la preuve de concept
PHASE 4 → Construire la solution
PHASE 5 → Tester & valider
PHASE 6 → Packager & documenter
PHASE 7 → Commercialiser
PHASE 8 → Maintenir & évoluer

Chaque phase a ses livrables, ses outils Python,
ses pièges et ses critères de passage à la suivante.

PHASE 1 – DÉTECTION DU PROBLÈME
"Voir ce que les autres subissent sans nommer."

1.1 LES 5 SOURCES DE DÉTECTION

SOURCE A – OBSERVATION DIRECTE (la plus fiable)
→ Tu travailles chez un client et tu vois une tâche
répétitive, manuelle, sujette à erreur
→ Tu entends "on fait ça à la main depuis des années"
→ Tu vois des fichiers Excel nommés
"final_v3_VRAIMENT_FINAL.xlsx"
→ Tu remarques qu'un rapport prend 2 jours à produire
et qu'il consiste à copier-coller des données

SOURCE B – DOULEURS VERBALISÉES
→ Forums (Reddit, StackOverflow, groupes LinkedIn)
→ Groupes métier Facebook / Slack / Discord
→ Commentaires YouTube sur des tutoriels
→ Questions récurrentes dans les groupes de ta niche
Signal : quand la même question revient + de 3 fois,
il y a un produit à créer.

SOURCE C – ANALYSE DE DONNÉES EXISTANTES
→ Le client te donne accès à ses données
→ Tu identifies des patterns qu'il ne voit pas
→ Churn non détecté, stock qui tourne mal,
segment rentable ignoré, coût caché non mesuré

SOURCE D – VEILLE CONCURRENTIELLE AUTOMATISÉE
→ Scraper les offres d'emploi de ta niche
(les offres d'emploi = problèmes non résolus)
→ Scraper les avis clients sur des outils SaaS
(les avis 3 étoiles = fonctionnalités manquantes)
→ Monitorer les forums de ta niche avec des alertes

SOURCE E – TES PROPRES FRICTIONS
→ Automatise d'abord ce qui t'énerve toi
→ Si ça t'énerve, ça énerve d'autres
→ C'est le meilleur prototypage possible :
tu es à la fois le développeur et l'utilisateur

1.2 OUTILS PYTHON POUR LA DÉTECTION

SCRAPING DE FORUMS ET OFFRES D'EMPLOI

```
# Détecter les douleurs sur Reddit
import praw # API Reddit officielle
```

```

reddit = praw.Reddit(
    client_id=os.getenv("REDDIT_ID"),
    client_secret=os.getenv("REDDIT_SECRET"),
    user_agent="problem-detector"
)

subreddit = reddit.subreddit("entrepreneur+freelance")
for post in subreddit.search("manual process OR excel OR
                             automation", limit=100):
    print(post.title, post.score, post.url)

ALERTES PAR MOT-CLÉ SUR UN SECTEUR


---


# Surveiller les mots-clés douloureux dans une niche
PAIN_KEYWORDS = [
    "hours every week", "manual", "copy paste",
    "no tool for", "wish there was", "anyone else tired of",
    "we do this manually", "takes forever"
]

VEILLE D'OFFRES D'EMPLOI (signaux de marché)


---


# Les offres d'emploi = problèmes non automatisés
# Scraper LinkedIn Jobs / Indeed par mots-clés
# Si 500 offres cherchent un "Excel reporting analyst"
# → il y a un outil à construire

```

1.3 JOURNAL DE DÉTECTION (à tenir à jour)

Problème observé	Source	Fréquence	Potentiel
			H / M / L
			H / M / L
			H / M / L

CRITÈRE DE PASSAGE → Phase 2 :
 Le problème est verbalisé par au moins 3 personnes
 différentes, dans au moins 2 sources différentes.

PHASE 2 – QUALIFICATION ET MESURE
 "Un problème non mesuré est une intuition, pas un marché."

2.1 LES 4 QUESTIONS DE QUALIFICATION

- Q1 – EST-CE DOULOUREUX ?
- Les gens cherchent-ils activement une solution ?
 - Combien de temps/argent ça leur coûte aujourd'hui ?
 - Un problème "nice to have" ne génère pas de revenus.
Un problème "must have" oui.
- Q2 – EST-CE FRÉQUENT ?
- Combien de personnes ou d'entreprises concernées ?
 - À quelle fréquence le problème se répète-t-il ?
 - Un problème rare + douloureux = mission sur-mesure
 - Un problème fréquent + modéré = outil / SaaS
- Q3 – EST-CE ADRESSABLE PAR DU CODE ?
- Le problème a-t-il une structure répétable ?
 - Les données d'entrée sont-elles numériques / digitales ?
 - La solution peut-elle être généralisée ou
est-elle trop spécifique à un contexte unique ?
- Q4 – Y A-T-IL DÉJÀ UNE SOLUTION ?
- Si oui : est-elle trop chère, trop complexe,
mal adaptée à une niche, ou mal connue ?
 - Un concurrent existant valide le marché.
L'absence de concurrent peut signifier
que le marché n'existe pas.

2.2 MESURER LE COÛT DU PROBLÈME

Faire remplir (ou estimer) :

Temps humain :

Tâche concernée	Temps/ semaine	Nb de personnes
	h	

Coût mensuel = Temps/semaine × 4 × Nb personnes
× Coût horaire moyen

Coût d'erreur : combien coûte une erreur humaine
sur ce processus ? (retour client, pénalité, reprise)

Coût d'opportunité : que pourrait faire cette personne
si elle ne faisait pas cette tâche répétitive ?

2.3 MODÈLES DE REVENUS POSSIBLES SELON LE PROBLÈME

Problème one-shot ponctuel

- Mission freelance fixe (audit + livraison)
- Tarif : 500€ – 5 000€ selon complexité

Problème récurrent (même client)

- Mission + maintenance mensuelle
- Tarif : forfait mensuel 200€ – 2 000€

Problème récurrent (marché entier)

- Script vendu en licence ou abonnement
- Template / outil no-code ou semi-technique
- SaaS minimal (Streamlit + base de données)

Problème de niche complexe

- Consulting + implémentation sur mesure
- Tarif journalier 400€ – 1 500€/j

CRITÈRE DE PASSAGE → Phase 3 :

Le coût du problème est quantifié.

Le modèle de revenu est choisi.

Au moins 1 personne a dit "j'en aurais besoin".

PHASE 3 – PROTOTYPE (PREUVE DE CONCEPT)

"La vitesse ici est une vertu. La perfection est un défaut."

3.1 OBJECTIF DU PROTOTYPE

- Prouver que la solution est faisable techniquement
- Obtenir un retour réel d'un utilisateur réel
- Ne PAS construire quelque chose de complet
- Ne PAS sur-architecturer
- Durée maximale : 1 à 3 jours

3.2 RÈGLES DU PROTOTYPE

INCLUDE :

- Le chemin principal (happy path) uniquement
- Les données réelles si possible, fictives sinon
- Un résultat visible par le client
(fichier, graphique, email, dashboard)

EXCLUDE :

- Gestion d'erreurs complète
- Tests unitaires
- Documentation
- Optimisation des performances
- Interface utilisateur polie

3.3 STRUCTURE D'UN PROTOTYPE EN PYTHON

prototype.py – script linéaire, sans architecture

```
# Objectif : montrer que ça marche. Rien d'autre.

import pandas as pd
import matplotlib.pyplot as plt

# ÉTAPE 1 – Charger les données
df = pd.read_csv("data_client.csv")
print(f"[OK] {len(df)} lignes chargées")

# ÉTAPE 2 – Transformation basique
df["date"] = pd.to_datetime(df["date"])
df_clean = df.dropna(subset=["montant", "client_id"])
print(f"[OK] {len(df_clean)} lignes valides")

# ÉTAPE 3 – Résultat visible
resume = df_clean.groupby("client_id")["montant"].sum()
resume.sort_values(ascending=False).head(10).plot(
    kind="bar", title="Top 10 clients par CA"
)
plt.tight_layout()
plt.savefig("output_prototype.png")
print("[OK] Graphique généré : output_prototype.png")
```

3.4 PRÉSENTER LE PROTOTYPE AU CLIENT

- Montrer le résultat, pas le code
- Formuler : "Voilà ce que ça donnerait avec vos vraies données"
- Demander : "Est-ce que c'est ça dont vous avez besoin ? Qu'est-ce qui manque ?"
- Écouter les retours avant d'écrire une ligne de plus

CRITÈRE DE PASSAGE → Phase 4 :

Le client (ou toi-même) a vu le prototype tourner.

Le résultat correspond à l'usage attendu.

Les ajustements nécessaires sont identifiés.

PHASE 4 – CONSTRUCTION DE LA SOLUTION

"Du prototype au produit : ajouter la robustesse, pas la complexité."

4.1 ORDRE DE CONSTRUCTION RECOMMANDÉ

SPRINT 1 – Cœur fonctionnel

- Réécrire le prototype en fonctions propres
- Ajouter la gestion des cas d'erreur critiques
- Ajouter le logging
- Tester sur données réelles

SPRINT 2 – Robustesse

- Gérer tous les cas limites identifiés
- Ajouter la validation des entrées
- Rendre le pipeline idempotent
- Configurer les paramètres via config.py / .env

SPRINT 3 – Livraison

- Automatiser (planification ou déclencheur)
- Ajouter la notification de résultat / d'échec
- Rédiger le README
- Tests sur l'environnement du client

4.2 PATTERNS DE SOLUTIONS PAR TYPE DE PROBLÈME

— PROBLÈME : RAPPORT MANUEL RÉCURRENT —————

Solution type :

Python (pandas) → transformation des données

→ génération automatique (Excel/PDF/email)

→ planification cron ou schedule

Librairies clés :

pandas, openpyxl, reportlab ou fpdf2,

smtplib, schedule ou cron

— PROBLÈME : SAISIE MANUELLE RÉPÉTITIVE —

Solution type :

Lecture source (CSV/API/BDD) → validation

→ écriture destination (BDD/API/Excel)

→ log des écritures + alertes d'anomalies

Librairies clés :

pandas, sqlalchemy, requests, loguru

— PROBLÈME : SURVEILLANCE / MONITORING —

Solution type :

Scraping ou appel API périodique → comparaison

avec état précédent → alerte si changement détecté

Librairies clés :

requests, beautifulsoup4 ou playwright,

schedule, smtplib ou slack-sdk

— PROBLÈME : ANALYSE DE DONNÉES NON EXPLOITÉES —

Solution type :

Extraction → nettoyage → analyse exploratoire

→ modèle ou règles métier → dashboard ou rapport

Librairies clés :

pandas, matplotlib/plotly, scikit-learn,

streamlit (pour le dashboard)

— PROBLÈME : TRAITEMENT DE DOCUMENTS —

Solution type :

Lecture PDF/Word/email → extraction structurée

→ normalisation → stockage ou action déclenchée

Librairies clés :

pdfplumber, python-docx, imaplib (emails),

re (regex), pandas

— PROBLÈME : INTÉGRATION ENTRE SYSTÈMES —

Solution type :

Lecture API source → transformation → écriture

API destination → log + gestion des conflits

Librairies clés :

requests, httpx, sqlalchemy, loguru

4.3 DÉCISIONS D'ARCHITECTURE À PRENDRE UNE FOIS

→ Script seul ou module avec src/ ?

→ Données en fichier ou en base de données ?

(< 100k lignes → fichier, > 100k lignes → BDD)

→ Interface ? (aucune / CLI / Streamlit / API Flask)

→ Déploiement local ou cloud ?

→ Qui maintient après livraison ?

CRITÈRE DE PASSAGE → Phase 5 :

Le pipeline tourne de bout en bout sur données réelles.

Les cas d'erreur principaux sont gérés.

Le logging produit un résumé lisible.

PHASE 5 – TEST ET VALIDATION

5.1 TYPES DE TESTS À CONDUIRE

TEST TECHNIQUE (toi)

→ Données vides → comportement correct ?

→ Format inattendu → erreur claire ou gestion ?

→ Volume extrême → tient en mémoire / en temps ?

→ Deux exécutions consécutives → résultat identique ?

→ Exécution sans internet → message d'erreur clair ?

TEST FONCTIONNEL (toi + client)

→ Le résultat correspond-il au besoin initial ?

→ Les chiffres correspondent-ils à ce que le client connaît manuellement ?

→ Le format de sortie est-il utilisable directement ?

TEST D'ACCEPTATION (client seul)

→ Le client peut-il lancer le script sans toi ?

→ Il comprend les logs et sait quoi faire en cas d'erreur ?

→ Il valide formellement le résultat ?

5.2 RECETTE – TABLEAU DE VALIDATION

Critère	Attendu	Obtenu / Statut
Nb lignes traitées		
Valeur agrégat clé 1		
Valeur agrégat clé 2		
Format fichier sortie		
Délai d'exécution	< X min	
Comportement si vide	log + exit	
Notification envoyée	oui	

→ Ce tableau est signé par le client.

Sans signature, la recette n'est pas faite.

CRITÈRE DE PASSAGE → Phase 6 :

Recette signée.

Aucun bug bloquant ouvert.

Le client a lancé le script seul une fois.

PHASE 6 – PACKAGING ET DOCUMENTATION

"Ce que tu livres, c'est une solution autonome.

Pas une dépendance à ta présence."

6.1 NIVEAUX DE PACKAGING SELON LA DESTINATION

LIVRAISON À UN CLIENT TECHNIQUE

→ Repo Git propre + README complet

→ requirements.txt + .env.example

→ Commentaires dans le code

→ Formation de 1h

LIVRAISON À UN CLIENT NON TECHNIQUE

→ Script avec interface CLI simple (click / argparse)
ou interface Streamlit basique

→ Instructions en langage naturel, pas en code

→ Vidéo de démonstration (Loom, 5 min max)

→ Document "que faire si..." pour les cas fréquents

PRODUIT RÉUTILISABLE (plusieurs clients)

→ Package Python installable (setup.py / pyproject.toml)

→ CLI avec options configurables

→ Documentation en ligne (MkDocs, Notion, README.md)

→ Versioning sémantique (1.0.0, 1.1.0, 2.0.0)

6.2 RENDRE UN SCRIPT UTILISABLE PAR UN NON-DÉVELOPPEUR

Interface CLI minimale avec click

```
import click
```

```
@click.command()
```

```
@click.option("--input", "-i", required=True,  
              help="Chemin du fichier CSV source")
```

```
@click.option("--output", "-o", default="output/",  
              help="Dossier de sortie (défaut: output/)")
```

```
@click.option("--email", "-e", is_flag=True,  
              help="Envoyer le rapport par email")
```

```
def main(input, output, email):
```

```
    """Traite le fichier source et génère le rapport."""
```

```
    run_pipeline(input, output, send_email=email)
```

Utilisation :

```
# python main.py --input ventes.csv --email
```

INTERFACE STREAMLIT (pour clients très non-techniques)

```
import streamlit as st
import pandas as pd

st.title("Analyse des ventes")
file = st.file_uploader("Charger votre fichier CSV")

if file:
    df = pd.read_csv(file)
    st.write(f"{len(df)} lignes chargées")
    result = transform(df)
    st.dataframe(result)
    st.download_button("Télécharger", result.to_csv(),
                       "resultat.csv")
```

6.3 CHECKLIST DE LIVRAISON

- [] README lisible par le client cible
- [] .env.example avec toutes les variables
- [] requirements.txt avec versions figées
- [] Logs du dernier run réussi fournis
- [] Formation faite et validée
- [] Contrat de maintenance ou clause fin de mission
- [] Facture finale envoyée avec le livrable

CRITÈRE DE PASSAGE → Phase 7 :

Un inconnu peut installer et lancer la solution
en suivant le README, sans te demander de l'aide.

PHASE 7 – COMMERCIALISATION

"Le problème résolu devient un actif.

Un actif se vend, se loue, ou se multiplie."

7.1 LES 4 FAÇONS DE MONÉTISER UNE SOLUTION PYTHON

MODÈLE A – MISSION SUR MESURE (le plus courant)

Tu résous le problème pour UN client.
Tarif : forfait fixe basé sur la valeur délivrée.

Comment pricer :
Valeur mensuelle du problème × 3 à 6 = prix de la mission
Exemple : problème coûte 2 000€/mois → mission à 6 000€
→ Le client est rentable dès le 3ème mois.

Upsell naturel : maintenance mensuelle après livraison.
Tarif maintenance : 10 à 20% du forfait initial / mois.

MODÈLE B – PRODUCTISER LA SOLUTION

Tu résous le même problème pour PLUSIEURS clients.
Tu construis une fois, tu vends plusieurs fois.

Formats :
→ Template Python vendu (Gumroad, GitHub, site)
→ Script clé en main avec configuration
→ Pack "solution + formation + documentation"
→ Prix : 97€ – 497€ par licence

Quand utiliser :
→ Quand tu as résolu le même problème 2 fois
→ Quand la personnalisation est < 20% du travail
→ Quand la niche est identifiable et joignable

MODÈLE C – ABONNEMENT / SAAS MINIMAL

Tu héberges la solution. Le client paye pour l'accès.

Stack minimale :
→ Script Python → résultats dans une base

- Interface Streamlit ou dashboard Metabase
- Hébergement : PythonAnywhere, Render, Railway
- Paiement : Stripe + facturation automatique

Prix : 49€ – 299€ / mois selon la valeur
 Avantage : revenu récurrent prévisible
 Inconvénient : support, uptime, maintenance

MODÈLE D – FORMATION ET CONSULTING

Tu vends ta méthode, pas seulement ton code.

Formats :

- Workshop "automatisez vos rapports Excel" (500€/j)
- Formation en ligne enregistrée (297€ one-shot)
- Consulting stratégique data (1 000€/j)
- Audit de pipeline (forfait 1 500€ – 3 000€)

7.2 CONSTRUIRE LE DISCOURS COMMERCIAL DE LA SOLUTION

STRUCTURE EN 4 TEMPS :

1. LE PROBLÈME (qu'ils reconnaissent)
 "Vous passez X heures par semaine à [tâche manuelle].
 Ça représente [coût] par mois et [risque d'erreur]."
2. LA CONSÉQUENCE (ce que ça coûte vraiment)
 "Pendant ce temps, votre équipe ne peut pas [valeur manquée]. Et chaque erreur coûte [coût d'erreur]."
3. LA SOLUTION (ce que tu fais)
 "J'automatise ce processus avec Python.
 En [délai], [résultat concret et mesurable]."
4. LA PREUVE (ce que tu as déjà fait)
 "J'ai fait ça pour [type de client similaire].
 Résultat : [gain temps/argent/fiabilité]."

7.3 OÙ ET COMMENT TROUVER LES CLIENTS

CANAU DIRECTS (les plus efficaces)

- LinkedIn : posts sur des problèmes résolus, pas sur des compétences techniques
- Réseau direct : anciens collègues, clients satisfaits
- Bouche-à-oreille : un client satisfait = 2 prospects

FORMAT DE CONTENU QUI ATTIRE

- "J'ai automatisé X en Y lignes de Python.
 Avant : Z heures/semaine. Après : 0. Voilà comment."
- Montrer le avant/après, pas le code
- Le client achète le résultat, pas la technique

CANAU INDIRECTS

- Malt, Upwork, Freelance.com (volume mais prix tirés)
- Groupes Facebook / LinkedIn de ta niche
- Partenariats avec des consultants non-techniques (ils ont les clients, tu as la solution)

7.4 PROPOSITION COMMERCIALE TYPE

Structure d'une proposition en 5 blocs :

BLOC 1 – Contexte et problème identifié

"Suite à notre échange du [date], voici ma compréhension de votre situation : [problème formulé en termes business, pas techniques]."

BLOC 2 – Solution proposée

"Je propose d'automatiser [processus] en [délai].
 Voici les étapes : [liste courte des livrables]."

BLOC 3 – Ce que ça change pour vous

"À la livraison : [résultat concret].
 En termes de temps économisé : [X heures/mois].
 En termes de fiabilité : [zéro erreur manuelle]."

BLOC 4 – Modalités

"Forfait : [montant€] TTC.
Acompte de 50% à la signature.
Solde à la recette validée.
Durée : [X semaines]."

BLOC 5 – Prochaine étape claire

"Pour démarrer, il me faut :
1. Votre accord par retour de mail
2. L'accès à [données / système]
3. Le versement de l'acompte"

PHASE 8 – MAINTENANCE ET ÉVOLUTION

"La livraison est le début de la relation,
pas sa conclusion."

8.1 CE QUI CASSE EN PRODUCTION (et quand)

Dans les 30 premiers jours :

- Format de données source qui change
- Credentials expirés ou modifiés
- Cas limites non testés qui apparaissent
- Volume qui dépasse ce qui était prévu

Dans les 3 à 6 mois :

- API tierce qui change de version
- Librairie Python mise à jour avec breaking change
- Besoin métier qui évolue (nouvelles colonnes, nouvelles règles, nouveau destinataire)

Après 1 an :

- Changement d'infrastructure (nouveau serveur, migration cloud)
- Remplacement d'un outil source (nouveau CRM, ERP)

8.2 CONTRAT DE MAINTENANCE – CE QU'IL DOIT COUVRIR

- Surveillance des logs et alertes
- Corrections de bugs post-livraison (délai garanti)
- Mises à jour librairies
- Adaptation aux changements de format source
- Hors périmètre : nouvelles fonctionnalités majeures

Tarif indicatif : 15 à 20% du forfait initial / mois

8.3 TRANSFORMER UN CLIENT EN REVENUS RÉCURRENTS

- Après chaque livraison, identifier le problème suivant
- Proposer l'audit comme entrée de relation systématique
- Chaque script livré = observation de 3 nouveaux problèmes potentiels
- Un client qui renouvelle vaut 5x un nouveau client (pas d'acquisition, pas de découverte)

8.4 CAPITALISER – CONSTRUIRE UNE BIBLIOTHÈQUE PRIVÉE

Après chaque mission, extraire :

- Les fonctions génériques réutilisables
- Les patterns de nettoyage courants
- Les connecteurs vers des systèmes communs
- Les templates de pipeline par type de problème

Organiser dans un repo privé :

```
ma_toolbox/  
├── connectors/      ← connexions BDD, APIs, fichiers  
├── transforms/     ← nettoyage, normalisation  
├── reporters/      ← génération PDF, Excel, email  
├── validators/     ← schémas, types, plages  
└── templates/      ← pipelines complets par type
```

- Chaque mission construit la suivante.
Au bout de 10 missions, ton temps de

développement est divisé par 3.

RÉCAPITULATIF – LA PIPELINE EN UN COUP D'ŒIL

PHASE 1	DÉTECTION Livvable →	Observer, scraper, écouter Journal de problèmes qualifiés
PHASE 2	MESURE Livvable →	Quantifier le coût, choisir le modèle Fiche problème avec ROI estimé
PHASE 3	PROTOTYPE Livvable →	Prouver en 1-3 jours que c'est faisable Script linéaire + retour client
PHASE 4	CONSTRUCTION Livvable →	Réécrire proprement, ajouter robustesse Pipeline structuré, logué, paramétré
PHASE 5	VALIDATION Livvable →	Tester les cas limites, recette signée Tableau de recette validé
PHASE 6	PACKAGING Livvable →	Documenter, rendre autonome README + formation + livraison
PHASE 7	VENTE Livvable →	Pricer, pitcher, proposer, conclure Proposition commerciale + contrat
PHASE 8	RÉCURRENCE Livvable →	Maintenir, capitaliser, étendre Bibliothèque privée + clients fidèles

LA VÉRITÉ FINALE DE CETTE PIPELINE

La valeur ne vient pas de ta capacité à coder.
Elle vient de ta capacité à voir un problème,
à le mesurer, à le résoudre proprement,
et à l'expliquer clairement à quelqu'un
qui ne comprend pas le code mais ressent la douleur.

Maîtrise cette pipeline – et Python devient
non pas un langage de programmation,
mais une machine à créer de la valeur.
